

Real-Time Tennis Action Recognition with NVIDIA TAO + Deepstream

Problem/Goal

Problem Statement

Recognizing tennis actions from raw video is challenging because strokes occur quickly, involve similar body motions, precise motions and require temporal context across multiple frames.

Project Goals

- Detect and classify tennis strokes accurately by training sequential vision models
- Perform continuous inference on live video streams
- Deploy the models using NVIDIA DeepStream for real-time performance
- Compare the performance of models

Tools

Nvidia TAO (Train, Adapt, Optimize) Toolkit

- SDK and CLI for Vision Models
- Allows you to train, fine-tune, and evaluate models
- Download pretrained models from ngc.nvidia.com
- .yaml spec files to adjust training
- python, C++, cuda
- Built on PyTorch and TensorFlow

Nvidia Deepstream

- Video analytics toolkit
- Allows you to build AI streaming pipelines for video, images, and audio
- Works with TAO models
- C++, cuda
- Built on GStreamer (Multi-media streaming framework)

THETIS (Three dimEnsional TennIs Shots) Dataset

Main features:

- captured using a Kinect device* (low quality)
- 8374 video sequences performed by 55 subjects, 31 beginners and 24 experts
- Only 1980 usable rgb .avi videos* (small dataset)
- 12 different classes of tennis actions: Backhand with two hands, Backhand, Backhand slice, Backhand volley, Forehand flat, Forehand open stands, Forehand slice, Forehand volley, Service flat, Service kick, Service slice, Smash.
- Scenes are indoor with multiple people in the background* (troublesome for bodypose3d)
- subject performs each activity 3 times



Approaches

BodyPose3D + PoseClassification

- Extract skeleton keypoints
- Fine-tune a classifier on skeleton graphs

ActionRecognitionNet

- Fine-tune end-to-end model on tennis videos

BodyPose3DNet Input/Output

Input

- Image

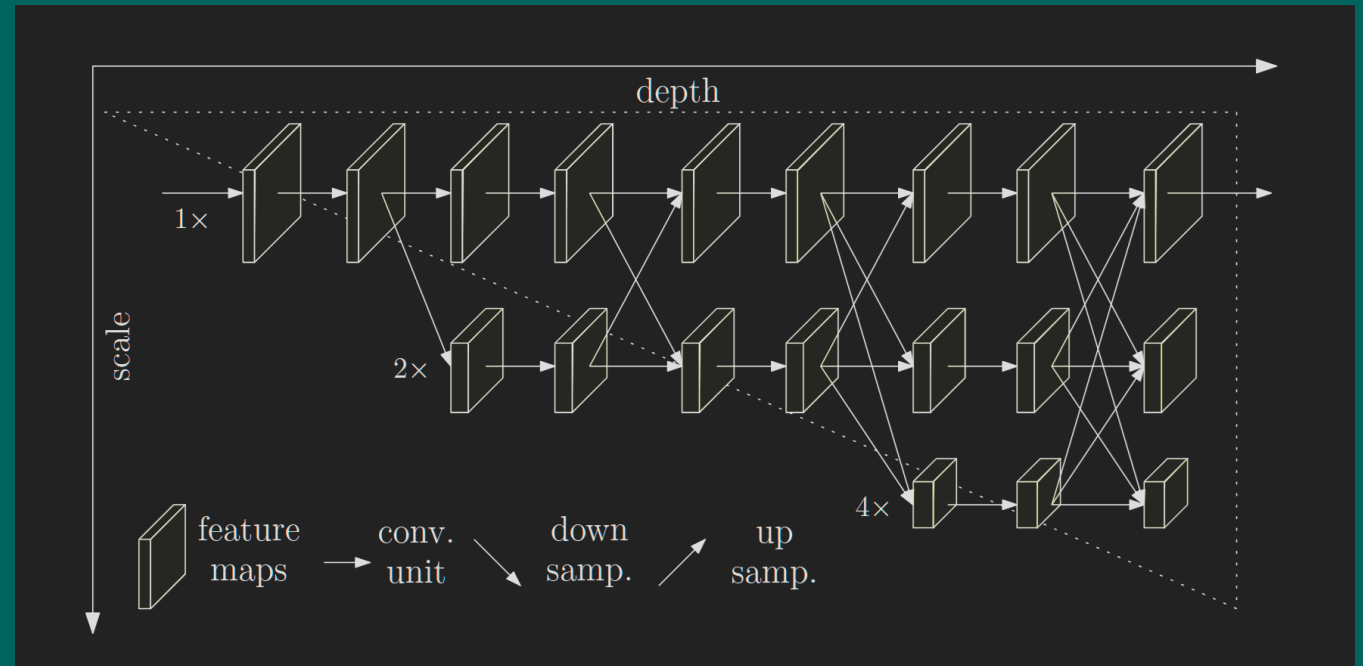
Output

- Keypoint locations (joint coordinates)
- $(x, y, z_rel, conf)$ for each joint
- (x, y) : pixel location in image
- z_rel : depth relative to pelvis
- $conf$: confidence score of joint coords



BodyPose3DNet Architecture

- High-res Net (HRNet) (CNN)
- Layers of subnetworks from high to low res
- Exchange information between subnetworks
- Spatial precision of high res + semantic representation of low res
- Good for pose estimation and semantic segmentation



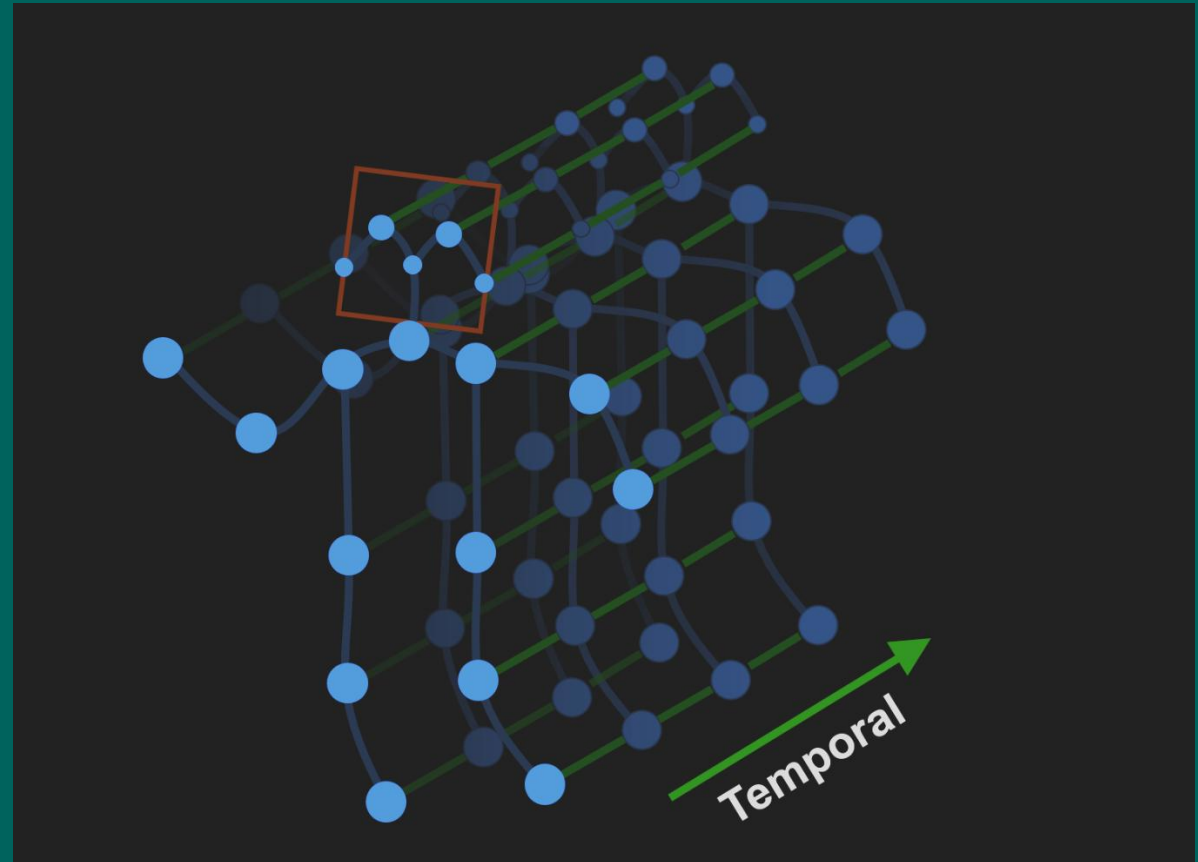
PoseClassificationNet

Input

- Graph
- Nodes: Joints
- Edges: Spatial + Temporal connections

Output

- Class label



PoseClassificationNet

Architecture:

- Backbone: Spatial Temporal Graph Convolutional Network (ST-GCN)
- Extension of Graph Convolutional Network, GCN – generalization of CNN

Key Points of ST-GCN:

- Performs convolution on a section of a graph instead of a section of an image
- Learns information about both the spatial and temporal relationships between joints

Linear Head + Sigmoid activation

Cross Entropy Loss

SGD

Results of Skeleton-based Action Recognition

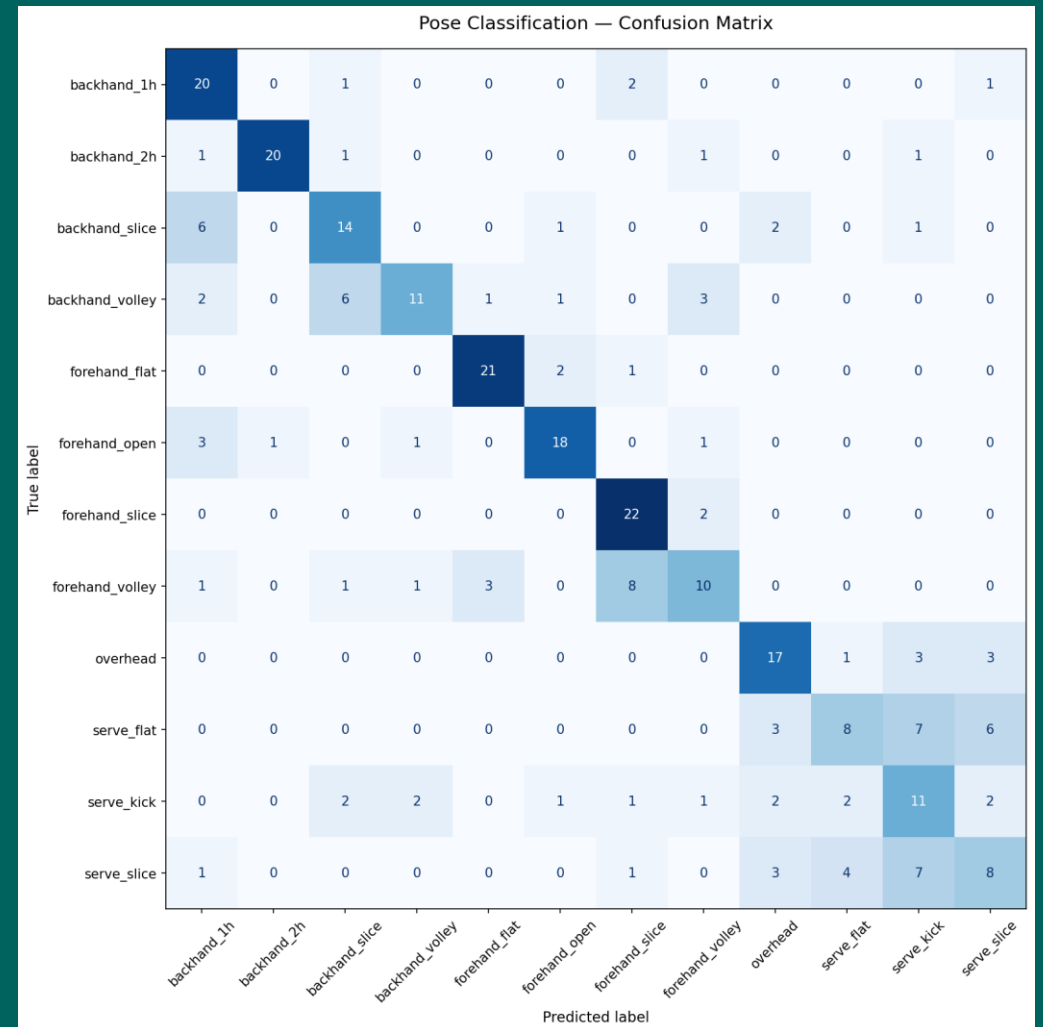
- Highest Average Class Accuracy: 62.5%
- Hard to distinguish

- Types of serves
- Volley vs. slice

Info About Best Model

- Fine-tuned on a backbone with 6 classes
- Lr=0.1
- No dropout
- 80 epochs, lr(50+)=0.01, lr(70+)=0.001 (prevents overfitting)
- Random split dataset (Beginners and experts mixed into train/test/val split)

Skeletons might not work well for tennis because you lose information about the racket movement



ActionRecognitionNet – Input/output

Input:

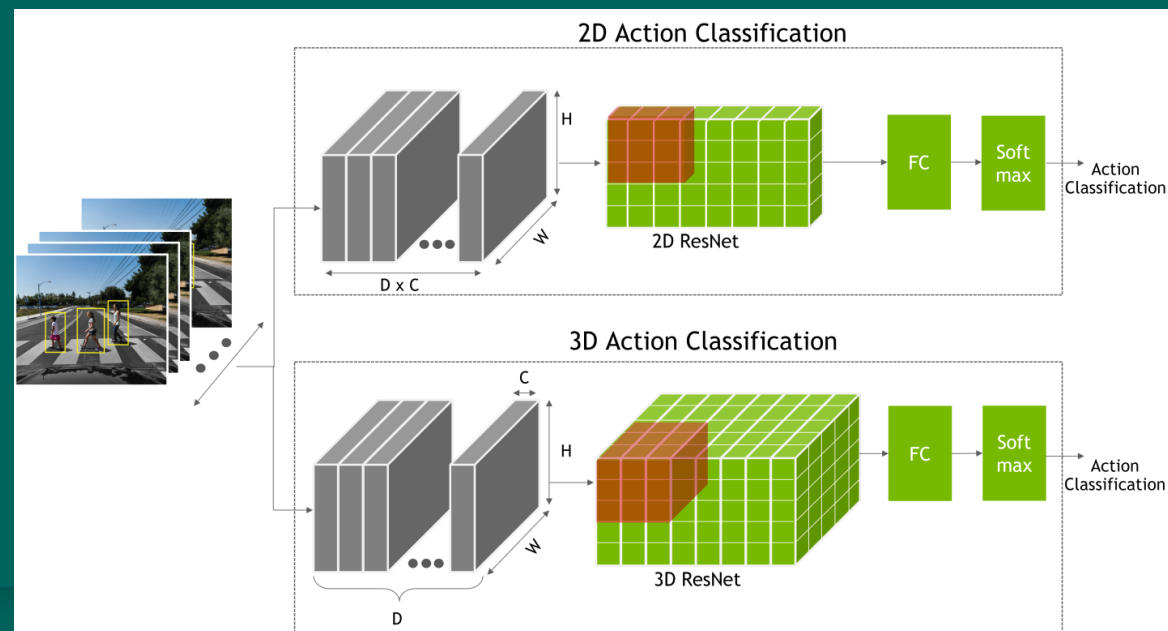
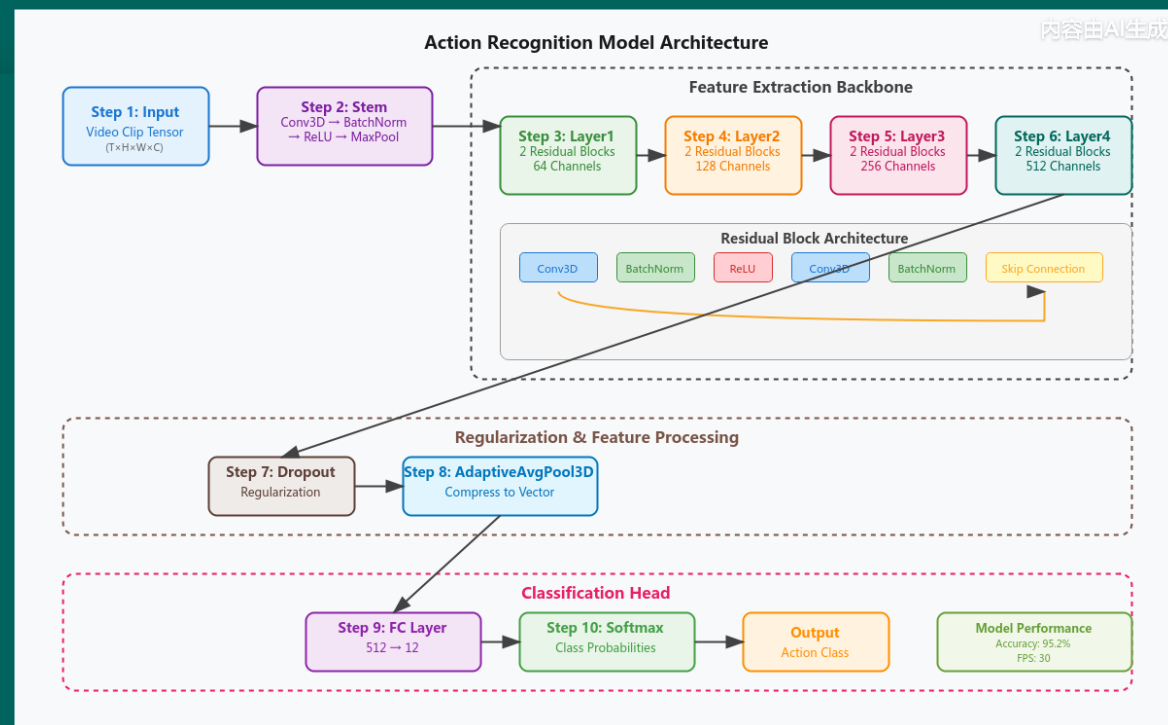
- input options were RGB-only input, optical flow (OF) only input, and two-stream joint input (RGB+OF), since the data set did not have optical flow we used RGB only input
- RGB + optical flow may improve performance because optical flow explicitly captures motion, but it requires additional preprocessing and was not used in this project.
- Input shape is implied to be (B,3,T,H,W) where B is batch size, channel = 3, sampled sequence length is T, H and W are the image dimensions.

Output:

- Outputs 12 class labels, classification problem
- Final classification layer according to cmd output is Linear(512 -> 12)
- Network outputs predicted label as well as the logits before softmax and probabilities of the label

Model Architecture

- Stem block is The first block of layers that prepares raw input before the main backbone and converts coarse spatial patterns to more meaningful feature maps
- Model uses strides >1 and pooling so resolution decrease while channels increase. So model focuses less on pixels and more on abstract high level features
- Adaptive avg pooling squashes $(B, 512, T, H, W)$ into $(B, 512, 1, 1, 1)$ by averaging across T, H, W for each channel.
- FC layer converts features into predictions or raw logits which become probabilities through the softmax
- Pretrained weights from a ResNet-18 3D CNN using RGB video clips, trained on the HMDB action dataset with 32-frame sequences was used.
- Transfer learning was used to train model.



Optimizer and loss functions

OPTIMIZER: Stochastic Gradient Descent (SGD) with Momentum

- Velocity update formula: $v = \beta v_t + \nabla J(\theta_t)$
- Weight update formula: $\theta_{t+1} = \theta_t - \alpha v_{t+1}$
- μ is the momentum coefficient and v_t is the accumulated velocity
- SGD with momentum smooths noisy gradient updates and improves convergence stability during training.

LOSS FUNCTION: Cross-Entropy Loss

- Formula: $L = - \sum_{i=1}^{12} y_i \log(\hat{y}_i)$
- Cross-entropy is standard for multi-class classification because it penalizes low probability assigned to the correct class.

Math: Residual network

3D convolution:

Formula: $y = F(x) + x$

Explanation: x is the original input, y is the expected block output, $F(x)$ is the output of the convolution block after inputting x , y is the sum of these two and what gets passed down the model.

- The paper hypothesizes that optimizing a residual mapping rather than the entire unreferenced mapping would be easier and more effective.
- Residual networks were design to resolve the degradation of training and test accuracy in deep networks.
- Authors hypothesized deep networks had troubling modelling identity functions.

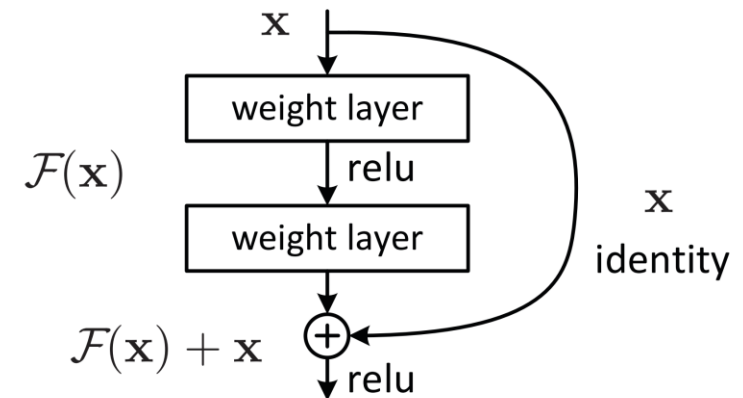


Figure 2. Residual learning: a building block.

Math: 3D CNN

3D convolution:

$$\text{Formula: } Y(c_o, t, h, w) = \sum_{c_i} \sum_{\tau} \sum_i \sum_j W(c_o, c_i, \tau, i, j) X(c_i, t + \tau, h + i, w + j)$$

Explanation: X is input tensor and W is kernel weights, c_o and c_i are output and input channel indices, T is time, i and j are the two spatial dimensions, just regular convolution but with 3d instead of 2d kernel. 3D convolution sees motion sequence which is pivotal in recognizing tennis moves compared to 2d convolutions.

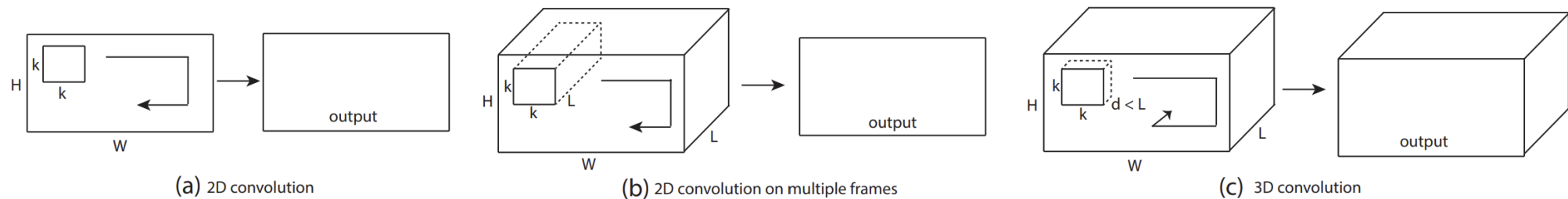


Figure 1. **2D and 3D convolution operations.** a) Applying 2D convolution on an image results in an image. b) Applying 2D convolution on a video volume (multiple frames as multiple channels) also results in an image. c) Applying 3D convolution on a video volume results in another volume, preserving temporal information of the input signal.

Data pt1

Run	Total Acc (%)	Batch Size	RGB Seq Length	Sample Rate	Learning Rate	Dropout	Clips_per_video	Lr_decay	Epochs	Initialization / Notes
experiment_transfer.yaml	75.98	16	32	1	0.0015	0.0	16	0.1	30	Baseline transfer run
experiment_transfer_sample_4.yaml	88.235	8	3	3	0.0005	0.10	8	0.1	14	4 th run with sample rate of 3 with rgb sequence length 3
experiment_transfer_sample_6.yaml	85.784	8	2	3	0.0005	0.10	8	0.1	14	6 th run of sample rate of 3 with rgb sequence length 2
experiment_transfer_sample_4_continue.yaml	88.725	8	3	3	0.0001	0.10	8	0.1	20	Continued from transfer 4 exp
experiment_transfer_sample_4_continue_2.yaml	89.706	8	3	3	0.0003	0.1	10	0.2	30	Continued from transfer 4 exp (trying to break through plateau)
experiment_transfer_sample_4_continue_5.yaml	90.686	8	3	3	0.0002	0.10	14	0.2	35	Best run, continued training from sample_4_continue_2

Data pt2

Class	Transfer	Sample_4	Sample_6	Continue	Continue_2	Continue_5
backhand	82.35	88.24	82.35	88.24	88.24	88.24
backhand2hands	88.24	94.12	94.12	94.12	94.12	94.12
backhand_slice	70.59	88.24	88.24	88.24	94.12	94.12
backhand_volley	52.94	100.0	88.24	100.0	100	100.0
flat_service	58.82	82.35	76.47	82.35	82.35	82.35
forehand_flat	76.47	82.35	94.12	94.12	88.24	94.12
forehand_openstands	94.12	100.0	94.12	100.0	100.0	100.0
forehand_slice	94.12	76.47	82.35	82.35	76.47	76.47
forehand_volley	82.35	94.12	94.12	88.24	94.12	94.12
kick_service	82.35	82.35	88.24	88.24	82.35	88.24
slice_service	76.47	88.24	82.35	88.24	94.12	94.12
smash	52.94	82.35	64.71	70.59	82.35	82.35
Total Accuracy	75.98	88.235	85.784	88.725	89.706	90.686

Data pt3

```
tensorboard --logdir /home/garywww/TAO/Training_results/results_y_8 --port 6006
```

```
tensorboard --logdir /home/garywww/TAO/Training_results/results_y_15 --port 6006
```

```
tensorboard --logdir /home/garywww/TAO/Training_results/results_y_18 --port 6006
```

Demo

- Explain tao toolkit (pytorch backend container models)
- Explain the yaml format (show it as well)
- Explain how we trained (pre-trained weights, checkpoints, how arrived at best result)
- Live demo

NVIDIA TAO Toolkit

- Framework for transfer learning and model optimization on NVIDIA GPUs
- Uses PyTorch backend for training
- Runs inside Docker containers for reproducibility
- Supports pretrained vision/video models
- YAML-driven experimentation
- Easy deployment to DeepStream / TensorRT

The background is a solid teal color. In the bottom-left corner, there is a large, dark teal abstract shape that resembles a hill or a wave. A lighter teal diagonal band runs from the top-left towards the bottom-right, passing behind the text.

Thank You